Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt


def apply_gaussian_filter(image, sigma):

    image = np.float32(image)

    frequency_domain = cv2.dft(image, flags=cv2.DFT_COMPLEX_OUTPUT)

    shifted_frequency_domain = np.fft.fftshift(frequency_domain, axes=(0, 1))

    rows, cols = image.shape
    crow, ccol = rows // 2, cols // 2

    mask = np.zeros((rows, cols), np.float32)

    for i in range(rows):
        for j in range(cols):
            mask[i, j] = np.exp(
                -((i - crow) ** 2 + (j - ccol) ** 2) / (2 * sigma ** 2)
            )

    # Apply Gaussian filter
    filtered_frequency_domain = shifted_frequency_domain * mask[:, :, np.newaxis]

    shifted_filtered_frequency_domain = np.fft.ifftshift(
        filtered_frequency_domain, axes=(0, 1)
    )

    filtered_image = cv2.idft(
        shifted_filtered_frequency_domain,
        flags=cv2.DFT_SCALE | cv2.DFT_REAL_OUTPUT
    )
    filtered_image = np.uint8(np.clip(filtered_image, 0, 255))
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
    return filtered_image

def apply_sharpening_filter(image, strength):

    image = np.float32(image)
    frequency_domain = cv2.dft(image, flags=cv2.DFT_COMPLEX_OUTPUT)

    shifted_frequency_domain = np.fft.fftshift(frequency_domain, axes=(0, 1))

    rows, cols = image.shape
    crow, ccol = rows // 2, cols // 2

    mask = np.zeros((rows, cols), np.float32)

    sigma = 20

    for i in range(rows):
        for j in range(cols):
            gaussian_mask = np.exp(
                -((i - crow) ** 2 + (j - ccol) ** 2) / (2 * sigma ** 2)
            )

            # High-boost filter
            mask[i, j] = 1 + strength * (1 - gaussian_mask)

    # Apply high-boost filter
    filtered_frequency_domain = shifted_frequency_domain * mask[:, :, np.newaxis]

    shifted_filtered_frequency_domain = np.fft.ifftshift(
        filtered_frequency_domain, axes=(0, 1)
    )

    filtered_image = cv2.idft(
        shifted_filtered_frequency_domain,
        flags=cv2.DFT_SCALE | cv2.DFT_REAL_OUTPUT
    )

    filtered_image = np.uint8(np.clip(filtered_image, 0, 255))

    return filtered_image
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
def extract_boundaries(image):

    boundary_image = cv2.Canny(image, 50, 150)

    return boundary_image

# Load Image 1
image1 = cv2.imread("IMAGE1.png", 0)

if image1 is None:
    raise FileNotFoundError("Image 1 not found. Check the image path.")

# Apply Gaussian smoothing
sigma = 20
smoothed_image = apply_gaussian_filter(image1, sigma)

# Apply High-Boost sharpening
strength = 1.5
sharpened_image = apply_sharpening_filter(image1, strength)

# Extract boundaries from sharpened image
boundary_image = extract_boundaries(sharpened_image)

plt.figure(figsize=(12, 9))

plt.subplot(2, 2, 1)
plt.imshow(image1, cmap='gray')
plt.title("Original Image 1")
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(smoothed_image, cmap='gray')
plt.title("Gaussian Smoothed Image")
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(sharpened_image, cmap='gray')
plt.title("High-Boost Sharpened Image")
plt.axis('off')
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
plt.subplot(2, 2, 4)
plt.imshow(boundary_image, cmap='gray')
plt.title("Binary Boundary Image")
plt.axis('off')

plt.tight_layout()
plt.show()

cv2.imwrite('Image1_Smoothed.png', smoothed_image)
cv2.imwrite('Image1_Sharpened.png', sharpened_image)
cv2.imwrite('Image1_Boundary.png', boundary_image)

print("Image 1 processing completed successfully.")
print("Smoothed image saved.")
print("Sharpened image saved.")
print("Boundary image saved.")
```

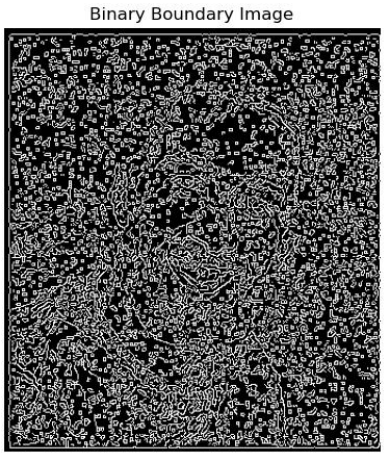
```
Image 1 processing completed successfully.
Smoothed image saved.
Sharpened image saved.
Boundary image saved.
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**









Observation

- Dense white and black speckles cover the original.
- Smoothing removes the noise but blurs the beard and eyes.
- Sharpening amplifies the noise, and the boundary map is full of noise edges.

Conclusion

- ✓ Gaussian removes speckle noise but costs sharpness.
- ✓ Do not sharpen a noisy image.
- ✓ Denoise first, then sharpen and detect edges.

```
# Load Image 2
image2 = cv2.imread("IMAGE2.png", 0)
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
if image2 is None:
    raise FileNotFoundError("Image 2 not found. Check the image path.")

# Apply Gaussian smoothing
sigma = 20
smoothed_image2 = apply_gaussian_filter(image2, sigma)

# Apply High-Boost sharpening
strength = 1.5
sharpened_image2 = apply_sharpening_filter(image2, strength)

# Extract boundaries from sharpened image
boundary_image2 = extract_boundaries(sharpened_image2)

plt.figure(figsize=(12, 9))

plt.subplot(2, 2, 1)
plt.imshow(image2, cmap='gray')
plt.title("Original Image 2")
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(smoothed_image2, cmap='gray')
plt.title("Gaussian Smoothed Image")
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(sharpened_image2, cmap='gray')
plt.title("High-Boost Sharpened Image")
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(boundary_image2, cmap='gray')
plt.title("Binary Boundary Image")
plt.axis('off')

plt.tight_layout()
plt.show()

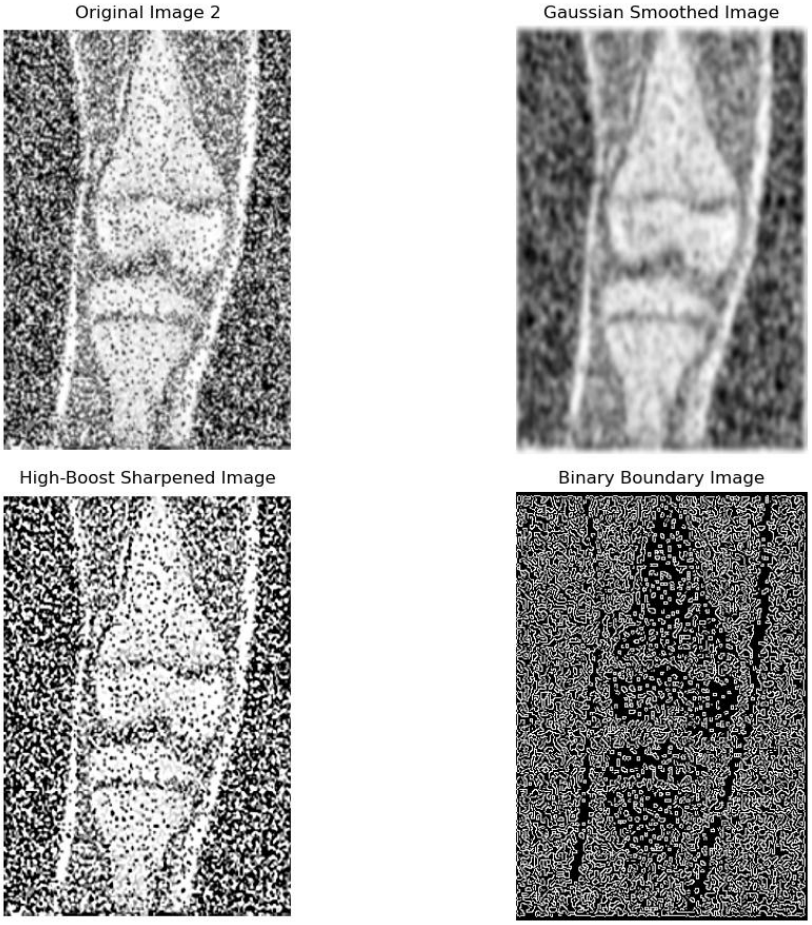
cv2.imwrite('Image2_Smoothed.png', smoothed_image2)
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
cv2.imwrite('Image2_Sharpened.png', sharpened_image2)
cv2.imwrite('Image2_Boundary.png', boundary_image2)

print("Image 2 processing completed successfully.")
print("Smoothed image saved.")
print("Sharpened image saved.")
print("Boundary image saved.")
```

```
Image 2 processing completed successfully.
Smoothed image saved.
Sharpened image saved.
Boundary image saved.
```



Observation

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

- Heavy speckle hides the bone structure.
- Smoothing makes the bone and joint gap visible but blurry.
- The sharpened and boundary images are dominated by noise.

Conclusion

- ✓ Smoothing is essential for this image.
- ✓ Sharpening and Canny on noisy data are unusable.
- ✓ A median filter would suit it better.

```
# Load Image 3
image3 = cv2.imread("IMAGE3.png", 0)

if image3 is None:
    raise FileNotFoundError("Image 3 not found. Check the image path.")

# Apply Gaussian smoothing
sigma = 20
smoothed_image3 = apply_gaussian_filter(image3, sigma)

# Apply High-Boost sharpening
strength = 1.5
sharpened_image3 = apply_sharpening_filter(image3, strength)

# Extract boundaries from sharpened image
boundary_image3 = extract_boundaries(sharpened_image3)

plt.figure(figsize=(12, 9))

plt.subplot(2, 2, 1)
plt.imshow(image3, cmap='gray')
plt.title("Original Image 3")
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(smoothed_image3, cmap='gray')
plt.title("Gaussian Smoothed Image")
plt.axis('off')

plt.subplot(2, 2, 3)
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
plt.imshow(sharpened_image3, cmap='gray')
plt.title("High-Boost Sharpened Image")
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(boundary_image3, cmap='gray')
plt.title("Binary Boundary Image")
plt.axis('off')

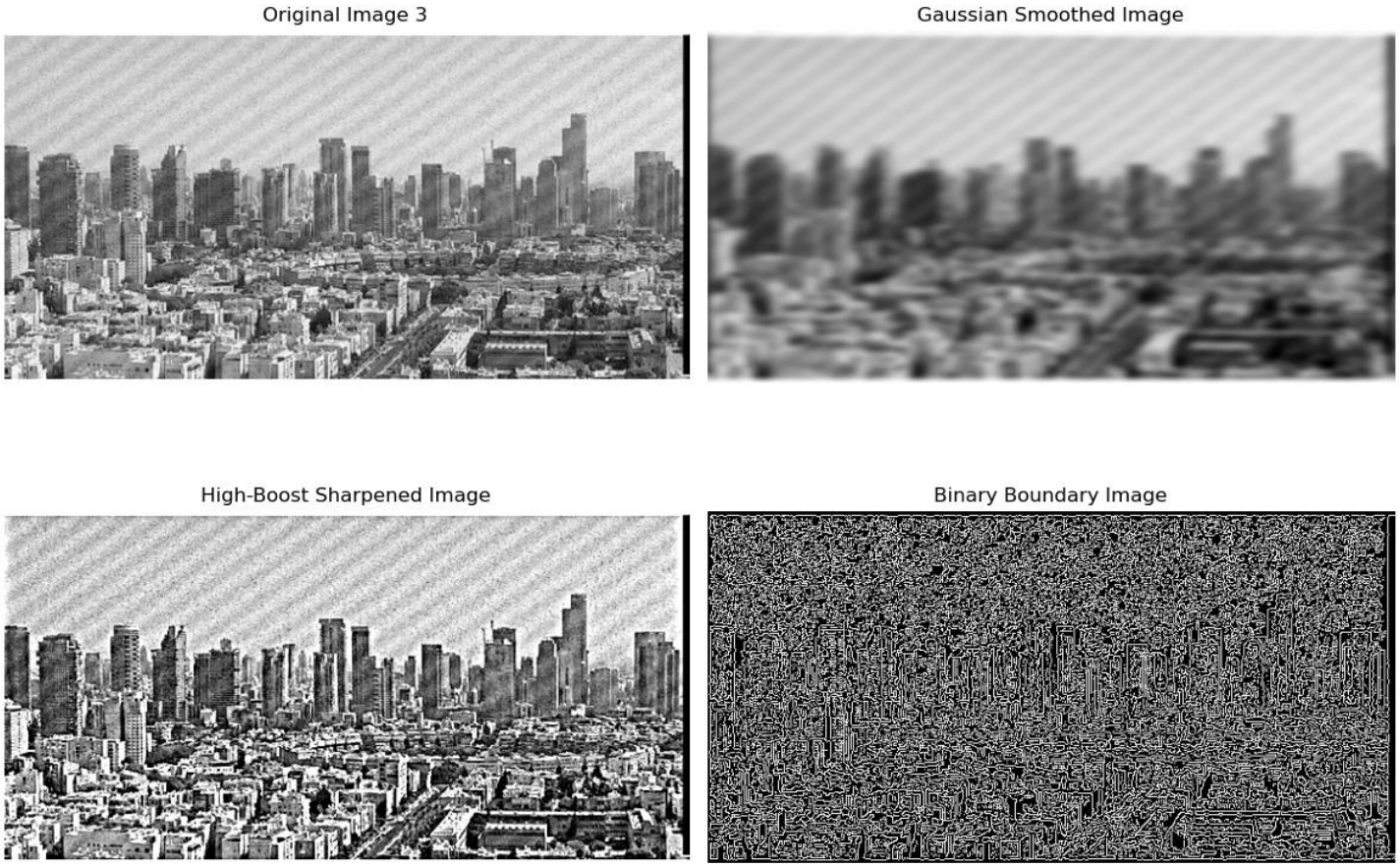
plt.tight_layout()
plt.show()

cv2.imwrite('Image3_Smoothed.png', smoothed_image3)
cv2.imwrite('Image3_Sharpened.png', sharpened_image3)
cv2.imwrite('Image3_Boundary.png', boundary_image3)

print("Image 3 processing completed successfully.")
print("Smoothed image saved.")
print("Sharpened image saved.")
print("Boundary image saved.")
```

```
Image 3 processing completed successfully.
Smoothed image saved.
Sharpened image saved.
Boundary image saved.
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**



Observation

- ➲ The original has diagonal stripes in the sky and fine grain.
- ➲ Smoothing blurs the buildings heavily, but the stripes remain.
- ➲ Sharpening shows building edges well but also strengthens the stripes and grain.

Conclusion

- ✓ A low-pass filter cannot remove low-frequency periodic stripes.
- ✓ A notch filter is the right tool.
- ✓ Sharpen only after removing the stripes.

```
# Load Image 4
image4 = cv2.imread("IMAGE4.png", 0)
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
if image4 is None:
    raise FileNotFoundError("Image 4 not found. Check the image path.")

# Apply Gaussian smoothing
sigma = 20
smoothed_image4 = apply_gaussian_filter(image4, sigma)

# Apply High-Boost sharpening
strength = 1.5
sharpened_image4 = apply_sharpening_filter(image4, strength)

# Extract boundaries from sharpened image
boundary_image4 = extract_boundaries(sharpened_image4)

plt.figure(figsize=(12, 9))

plt.subplot(2, 2, 1)
plt.imshow(image4, cmap='gray')
plt.title("Original Image 4")
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(smoothed_image4, cmap='gray')
plt.title("Gaussian Smoothed Image")
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(sharpened_image4, cmap='gray')
plt.title("High-Boost Sharpened Image")
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(boundary_image4, cmap='gray')
plt.title("Binary Boundary Image")
plt.axis('off')

plt.tight_layout()
plt.show()

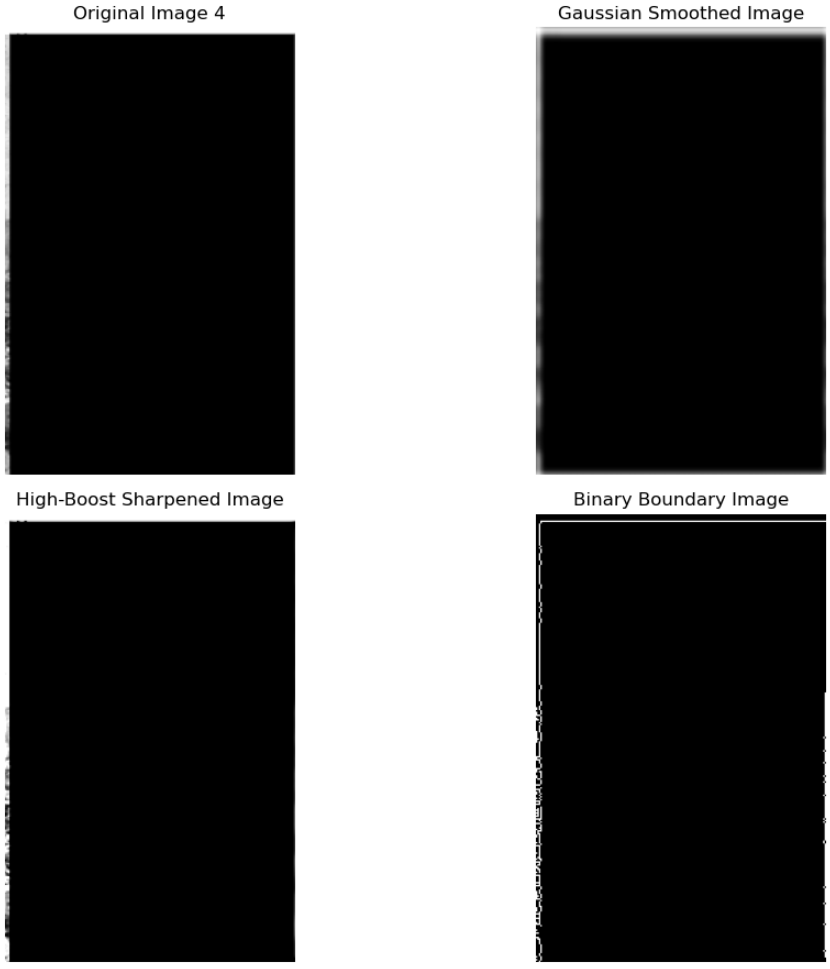
cv2.imwrite('Image4_Smoothed.png', smoothed_image4)
cv2.imwrite('Image4_Sharpened.png', sharpened_image4)
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
cv2.imwrite('Image4_Boundary.png', boundary_image4)

print("Image 4 processing completed successfully.")
print("Smoothed image saved.")
print("Sharpened image saved.")
print("Boundary image saved.")
```

```
Image 4 processing completed successfully.
Smoothed image saved.
Sharpened image saved.
Boundary image saved.
```



Observation

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

- The image is almost entirely black, with a faint strip on the left edge.
- Smoothing and sharpening change almost nothing.
- The boundary map shows only the outer frame.

Conclusion

- ✓ Filtering cannot recover detail from a very low dynamic range.
- ✓ Apply histogram equalization or gamma correction first.
- ✓ Then re-run smoothing, sharpening and edge detection.

```
# Load Image 5
image5 = cv2.imread("IMAGE5.png", 0)

if image5 is None:
    raise FileNotFoundError("Image 5 not found. Check the image path.")

# Apply Gaussian smoothing
sigma = 20
smoothed_image5 = apply_gaussian_filter(image5, sigma)

# Apply High-Boost sharpening
strength = 1.5
sharpened_image5 = apply_sharpening_filter(image5, strength)

# Extract boundaries from sharpened image
boundary_image5 = extract_boundaries(sharpened_image5)

plt.figure(figsize=(12, 9))

plt.subplot(2, 2, 1)
plt.imshow(image5, cmap='gray')
plt.title("Original Image 5")
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(smoothed_image5, cmap='gray')
plt.title("Gaussian Smoothed Image")
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(sharpened_image5, cmap='gray')
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
plt.title("High-Boost Sharpened Image")
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(boundary_image5, cmap='gray')
plt.title("Binary Boundary Image")
plt.axis('off')

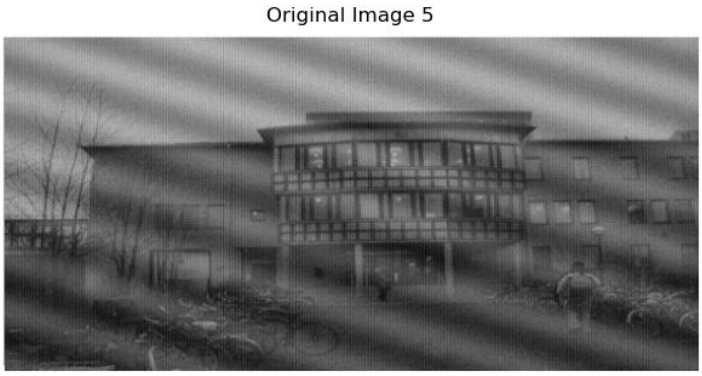
plt.tight_layout()
plt.show()

cv2.imwrite('Image5_Smoothed.png', smoothed_image5)
cv2.imwrite('Image5_Sharpened.png', sharpened_image5)
cv2.imwrite('Image5_Boundary.png', boundary_image5)

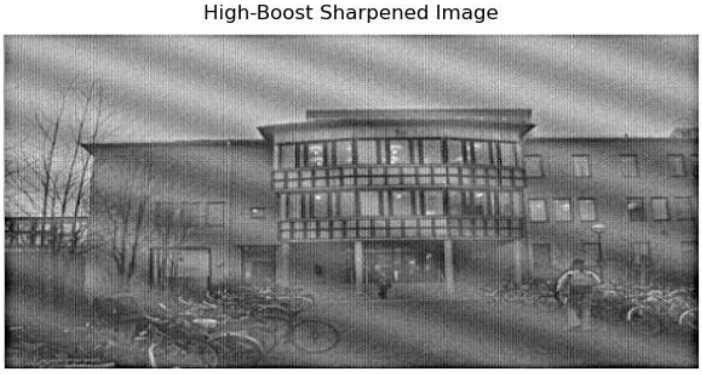
print("Image 5 processing completed successfully.")
print("Smoothed image saved.")
print("Sharpened image saved.")
print("Boundary image saved.")
```

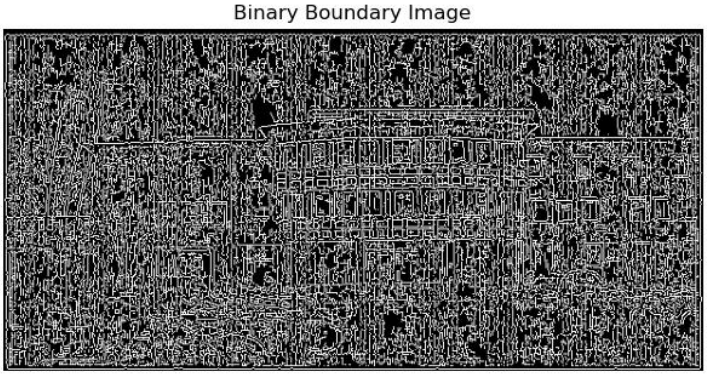
```
Image 5 processing completed successfully.
Smoothed image saved.
Sharpened image saved.
Boundary image saved.
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**









Observation

- ➲ The original has wide diagonal bands, texture noise and low contrast.
- ➲ Smoothing blurs the windows badly, and the bands stay.
- ➲ Sharpening clarifies windows and bicycles, but the bands and clutter remain.

Conclusion

- ✓ This is periodic interference, as in Image 3, so low-pass fails.
- ✓ A notch filter is needed.
- ✓ High-Boost is the better of the two filters used here.

```
# Load Image 6
image6 = cv2.imread("IMAGE6.png", 0)
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
if image6 is None:
    raise FileNotFoundError("Image 6 not found. Check the image path.")

# Apply Gaussian smoothing
sigma = 20
smoothed_image6 = apply_gaussian_filter(image6, sigma)

# Apply High-Boost sharpening
strength = 1.5
sharpened_image6 = apply_sharpening_filter(image6, strength)

# Extract boundaries from sharpened image
boundary_image6 = extract_boundaries(sharpened_image6)

plt.figure(figsize=(12, 9))

plt.subplot(2, 2, 1)
plt.imshow(image6, cmap='gray')
plt.title("Original Image 6")
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(smoothed_image6, cmap='gray')
plt.title("Gaussian Smoothed Image")
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(sharpened_image6, cmap='gray')
plt.title("High-Boost Sharpened Image")
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(boundary_image6, cmap='gray')
plt.title("Binary Boundary Image")
plt.axis('off')

plt.tight_layout()
plt.show()

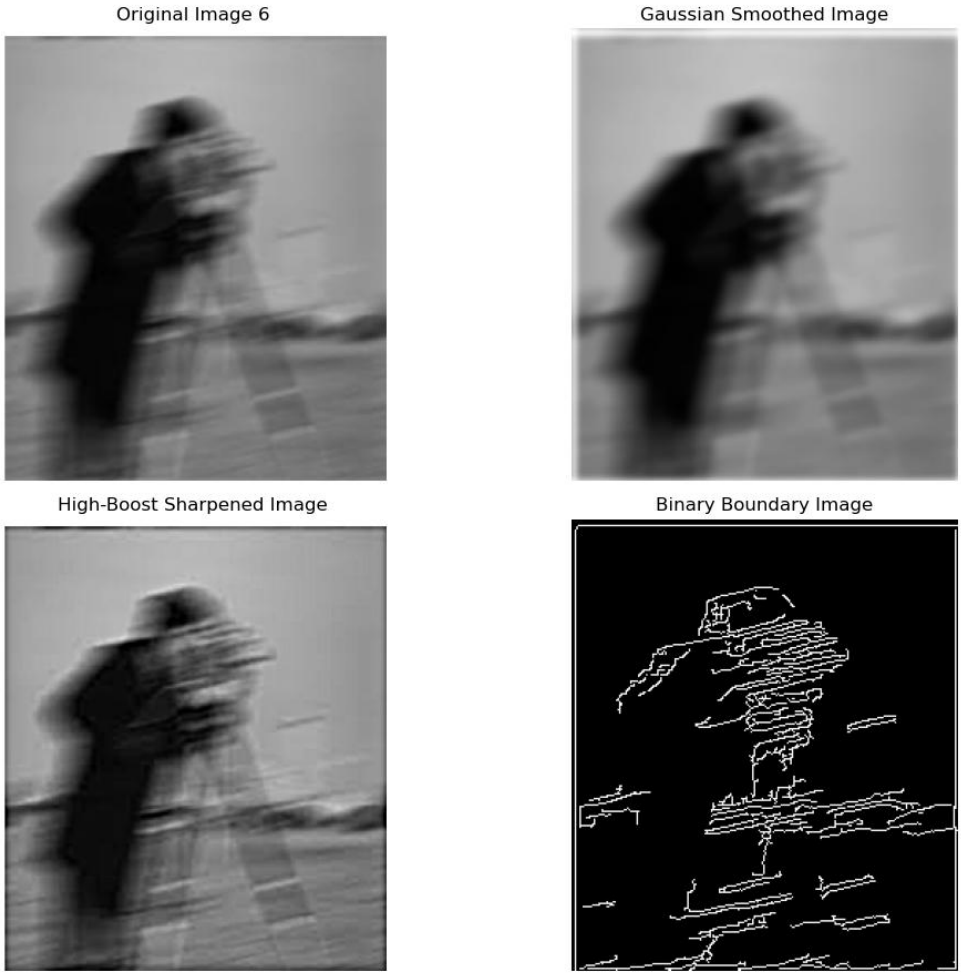
cv2.imwrite('Image6_Smoothed.png', smoothed_image6)
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

```
cv2.imwrite('Image6_Sharpened.png', sharpened_image6)
cv2.imwrite('Image6_Boundary.png', boundary_image6)

print("Image 6 processing completed successfully.")
print("Smoothed image saved.")
print("Sharpened image saved.")
print("Boundary image saved.")
```

```
Image 6 processing completed successfully.
Smoothed image saved.
Sharpened image saved.
Boundary image saved.
```



Observation

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

- ➲ Strong horizontal motion blur smears the person and tripod.
- ➲ Smoothing makes the blur worse.
- ➲ Sharpening gives crisper outlines, and the boundary map is the cleanest of all six.

Conclusion

- ✓ High-Boost works best because there is no noise to amplify.
- ✓ Smoothing is the wrong choice for a blurred image.
- ✓ Wiener or inverse deblurring would restore it properly.

QUALITATIVE COMPARISON

- ❖ Smoothing cuts sharpness by about 40–80% on noisy images (1, 2, 3, 5).
- ❖ High-Boost raises sharpness 2 to 4 times, mostly by amplifying noise in Images 1, 2 and 3.
- ❖ A high edge density (22–32%) means noise edges, not real structure.
- ❖ Image 6 gives the most meaningful edge map.
- ❖ For Images 4 and 6 the “smoothed” sharpness is inflated by border artifacts.

QUALITATIVE COMPARISON OF FILTER CHOICES

Gaussian low-pass ($\sigma = 20$)

- o Good for random noise (Images 1, 2).
- o Blurs detail, cannot remove stripes (Images 3, 5), and worsens blur (Image 6).

High-Boost (strength 1.5)

- o Good for clean or blurred images (Image 6).
- o Amplifies noise (Images 1, 2) and does not fix low contrast (Image 4).

Canny (50, 150)

- o Good on clean, sharp images (Image 6).
- o Gives false edges on noisy images and finds almost nothing on dark ones.

Better-suited filters

- o Median filter for salt-and-pepper noise.
- o Notch filter for periodic stripes.
- o Histogram equalization for the dark image.
- o Wiener filter for motion blur.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim: To apply smoothing, sharpening and edge detection techniques to six degraded images and compare their results.**	
Open-ended-Problem: 10	**Date:04-11-2026**	**Enrollment No:92400133121**

OVERALL CONCLUSION

- No single filter works for all images, so the choice depends on the type of degradation.
- Gaussian removes random noise but blurs and cannot remove periodic noise.
- High-Boost sharpens edges but amplifies noise, so denoise first.
- Canny works well only on clean, well-contrasted images.
- The best pipeline is de-noise, then fix contrast, then sharpen, then detect edges.